



VIT

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)



Test : CAT- I / CAT- II / Mid-Term

Register No.: 22BCB0001 Course Code: BCSE2024 Class NBR: 6331
Student's Name: Mr/Ms. K. Geethika Varshini Course Title: Data Structures & Algorithms
Programme : BTECH School : SCOPE Faculty Name : Prof. : Priyanka N
Slot : A1 Session : FN/AN/EVN Date of Exam : 26/3/23 No. of pages written :

K. Geethika
Signature of the Student

Signature of the Invigilator : P. Priyanka
Name / Emp. ID of the Invigilator : 17338

Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks	
	a	b		a	b		a	b		a	b		a	b
1	4	2	9			17			25			33		
2	3	5	10			18			26			34		
3	8		11			19			27			35		
4	8		12			20			28			36		
5	7		13			21			29			37		
6			14			22			30			38		
7			15			23			31			39		
8			16			24			32			40		
Sub-Total	30	7	Sub-Total			Sub-Total			Sub-Total			Sub-Total		

Grand Total Marks :

37

Marks In Words : THREE SEVEN

Evaluator's Signature : P. Priyanka

Date :

Evaluator's Name : Prof. PRIYANKA N

1) a) i) $T(n) = 3T\left(\frac{n}{2}\right) + n^2$

Master's theorem: $aT(n/b) + f(n)$

$$f(n) = n^k \log^p n$$

$$a = 3$$

$$b = 2$$

$$\log_b a = \log_2 3 = 1.5$$

$$f(n) = n^2 \log^0 n$$

$$k = 2, p = 0$$

$$\log < k, \text{ so}$$

$$\text{if } p \geq 0 = O(n^k \log^p n)$$

$$p \leq 0 = O(n^k)$$

$$\text{As, } p = 0$$

$$O(n^2 \log^0 n)$$

ii) $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$

Master's theorem: $aT(n/b) + f(n)$

$$f(n) = n^k \log^p n$$

$$a = 2$$

$$b = 2$$

$$\log_b a = \log_2 2 = 1$$

$$f(n) = n^1 \log^1 n$$

$$k = 1, p = 1$$

Here, $\log = k$

So,

$$\text{if } \log = k$$

$$p > -1 = O(n^k \log^{1+p} n)$$

$$p = -1 = O(n^k \log \log n)$$

$$p < -1 = O(n^k)$$

$$\text{As, } p = 1$$

$$O(n^k \log^2 n)$$

b) ii) $T(n) = \begin{cases} T\left(\frac{n}{2}\right) + 1, & \text{for } n > 1 \\ 1, & \text{for } n = 1 \end{cases}$

$$T(n) = T\left(\frac{n}{2}\right) + 1 \quad \text{--- ①}$$

$$T(n/2) = T\left(\frac{n}{4}\right) + 1 \quad \text{--- ②}$$

sub eq ② in eq ①

$$T(n) = T\left(\frac{n}{4}\right) + 1 + 1$$

$$T(n) = T\left(\frac{n}{4}\right) + 2$$

$$T(n) = T\left(\frac{n}{8}\right) + 3$$

$$T(n) = T\left(\frac{n}{16}\right) + 4$$

$$\frac{1}{2^k}$$

$$k$$

$$2^k = n$$

$$k \log_2 2$$

$$T(n) = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \frac{n}{16} + \dots$$

$$= n \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots \right)$$

$2^K = n$
log on both sides

$$K \log_2 = \log n$$

$$K = \log n$$

$$O(n) = O(n \log n)$$

3. $(a/(b-c+d)) * (e-a) * c$

Input	Stack	Postfix / output
(	(	
a	(	a
/	(/	a
(	((	a
b	((	ab
-	((-	ab
c	((-	abc
+	((-+	abc
d	((-+	abcd
)	((	abcd+-
)	(	abcd+ - /

4

*	*	abcd+ - /
(	*(	abcd+ - /
e	*(	abcd+ - /e
-	*(-	abcd+ - /e
a	*(-	abcd+ - /e a
)	*	abcd+ - /e a -
*	**	abcd+ - /e a -
c	**	abcd+ - /e a - c

Result $\rightarrow$ abcd+ - /e a - c **

Algorithm

- $\rightarrow$ When an operand comes we push the operand into the postfix
- $\rightarrow$ When an operator comes we push the operator into the stack
- $\rightarrow$ When an operator has ^{higher} lower precedence than top operator in the stack, we push the operator into stack
- $\rightarrow$ When an operator has same precedence then of top operator in stack, we push the operator into the stack

5



→ When the operator has lower precedence than the top operator in the stack. we pop the operator present in stack until we get operator of same precedence or lower precedence

eg: Here in this question we can see stack has $(((-$ & $+$ operator is added & the stack becomes $(((-+$ becoz they have same precedence

→ When ever we see an open parenthesis ' $($ ' we push it into the stack

{	$\wedge$ - higher precedence
	$\ast / \%$ - medium precedence
	$+, -,$ lower precedence

→ When ever we see an closed parenthesis we pop all the operators present in the stack to postfix

eg: Here in the above question we can see stack has $(((-+ & ')$ closed parenthesis is added, so we pop the operators present in the open parenthesis & becomes $(($

→ At the end when operators are left in the stack we pop all the operators & we can place them in final ending of equation.

4. a) Advantages of circular queue

→ As circular queue helps in efficient memory storage.

→ It takes or utilizes lesser time than linear queue.

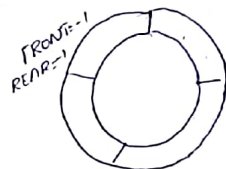
→ In linear queue once an element is deleted the memory utilised by the deleted element cannot be used again by new element i.e., we cannot add new element to the location of deleted element. but in circular queue we can add new element & we can even push move the elements

→ The linear queue is in linear form & the circular queue is in the form of circle as the circular queue is in the form of circle it takes less time than linear queue.

```
6) {  
    For  
        REAR = MAX - 1  
        set OVERFLOW  
  
    If  
        REAR == -1, FRONT == -1  
        SET REAR = FRONT = 0  
    }  
  
    else if  
        REAR != 0, FRONT != 0 - 1  
        SET REAR = 1 + REAR  
  
    else  
        SET VAL = enqueue[REAR]
```

1) initial we have to check whether the queue is empty. If it is full is overflow.

→ As, $N=4$, circular queue



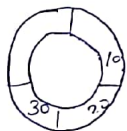
→ Initially insert the element 10, $front=0$, $rear=0$



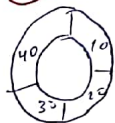
→ then insert the element 20, $front=0$, $rear=1$



→ then insert the element 30, $front=0$, $rear=2$



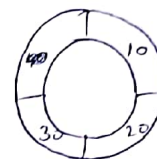
→ then insert the element 40, $front=0$, $rear=3$



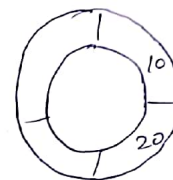
2. Deletion of two elements

→ we delete the elements from front end.

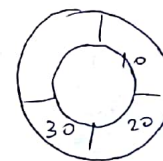
$rear=3$,
 $front=1$



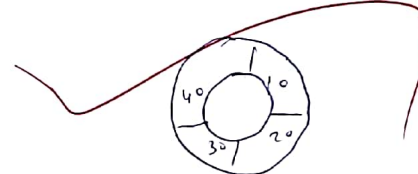
→ then we delete other element, $rear=3$, $front=2$



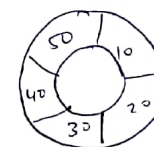
3. Insert the element 30, $rear=4$, $front=2$



insert the element 40, $rear=5$, $front=2$

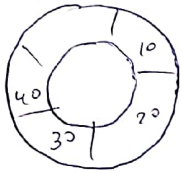


Here are going to insert other extra element 50
 $rear=6$, $front=2$

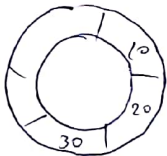


4. Successive deletion of three elements

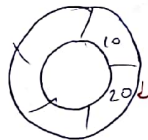
→ First we delete the top element in front end. Rear = 6, front = 3



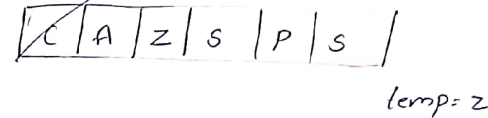
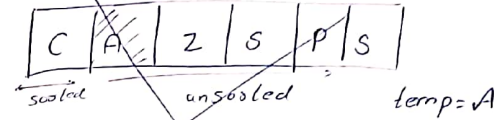
→ then we delete the preceding element in front end. Rear = 6, front = 4



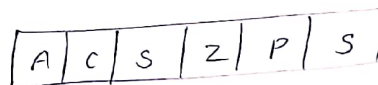
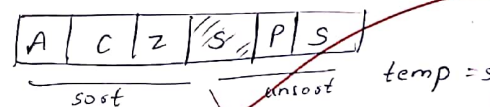
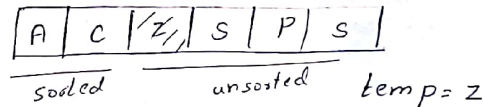
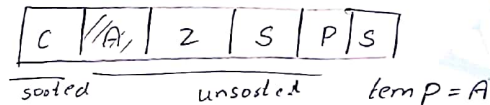
→ then we delete the next preceding element
Rear = 6, front = 5



5. Insertion sort



5. Insertion sort



A	C	S	Z	P	S
---	---	---	---	---	---

sort unsort temp = P

A	C	S	?	Z	S
---	---	---	---	---	---

A	C	P	S	Z	S
---	---	---	---	---	---

A	C	P	S	Z	S
---	---	---	---	---	---

sort unsort temp = S

A	C	P	S	S	Z
---	---	---	---	---	---

```
for (i=0, i<n-1, i++)
{
    int min = i;
```

```
    for (j=i+1, j<n, j++)
```

```
    while
    if (Array[a] < min[i])
```

```
        min[j]
```

```
    if (Array[a] > min[i])
```

```
    {
        swap (min, a) values
```

```
    }
```

```
}
```

```
return 0
```

```
}
```

12

2. b) $f(n) = 3n+2$

in big-O $0 > f(n) > g(n)$

$0 > 3n+2 > 3n+2n$

$0 > 3n+2 > 3n+2n$, if $n=1$

$0 > 5 >$

$0 > 3n+2 > 3n+4n$

if $n=1$

$0 > 3n+2 > 7$

if satisfy the above condition

$f(n) = O(g(n))$

$0 > f(n) > c \cdot g(n)$

$c = 2$

$\& n_0 = 1$

$\therefore$ The values of $c=2$

and the value of $n_0=1$

13



2. a) i) $sum = 0$ — 1

for ($k = 1; k \leq n; k = k * 2$) $\rightarrow (n+1)$

for ($j = 1; j \leq n; j++$) $\rightarrow n(n+1)$

$sum++ \quad n \times n$

$k \quad 1 \quad 2 \quad 4 \quad 8 \quad 16 \quad \rightarrow 2^k$

$j \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad \rightarrow k$

$n \quad \text{of} \quad n = 2^k$
 $\log n = k \log_2 2$
 $k = \log n \quad \therefore O(\log n)$

b) ii) $sum = 0;$

for ($j = 1; j \leq n; j++$)

for ($i = 1; i \leq j; i++$)

$sum++;$

for ($k = 0; k \leq n; k++$)

$a[k] = k$

$i \quad 1 \quad 2 \quad 3 \quad - \quad - \quad - \quad n$

$j \quad 1 \quad 2 \quad 3 \quad - \quad - \quad - \quad n$

$k \quad 0 \quad 1 \quad 2 \quad - \quad - \quad - \quad n-1$

$\therefore O(n^3)$